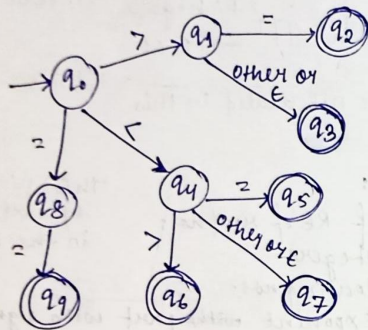


(4) Relational operators:

RE: $\langle \langle \rangle \rangle = \langle \rangle$

PA:

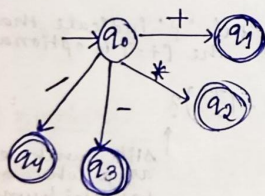


operators:

RE: + | * | - | /

→ наш оператор не рече '1' наведна

FA:



Ques:

Ques: $G: S \rightarrow SS + [SS *] a$

find LR(0) and SLR(1)

$$G': S' \rightarrow S$$
$$\begin{aligned} S &\rightarrow SS+ \\ S &\rightarrow SS \end{aligned}$$

$$I_0 = \begin{cases} S' \rightarrow \underline{S} \\ S \rightarrow \underline{SS} + \\ S \rightarrow \underline{S} S * \\ S \rightarrow \underline{a} \end{cases}$$

$$I_1 = \begin{cases} S' \rightarrow S, \\ S \rightarrow S, \underline{S} + \\ S \rightarrow S, \underline{S} * \\ S \rightarrow S, \underline{SS} + \\ S \rightarrow S, \underline{SS} * \\ S \rightarrow S, \underline{a} \end{cases}$$

$$I_2 = \{ S \rightarrow a, \}$$

$$I_3 = \begin{cases} S \rightarrow SS, \underline{+} \\ S \rightarrow SS, \underline{*} \\ S \rightarrow S, \underline{S} + \\ S \rightarrow S, \underline{S} * \\ S \rightarrow S, \underline{SS} + \\ S \rightarrow S, \underline{SS} * \\ S \rightarrow \underline{a} \end{cases}$$

$$I_4 = \begin{cases} S \rightarrow SS +, \\ S \rightarrow SS *, \end{cases}$$

$$I_5 = \{ S \rightarrow SS +, S \rightarrow SS *, \}$$

LR(0) table

state no.	action				GoTo
	α	$*$	$+$	$\$$	
0	S2				1
1	S2				3
2	r3	r3	r3	r3	
3	S2				3
4	r1	r1	r1	r1	
5	r2	r2	r2	r2	

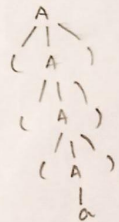
The table for $SLR(1)$ will also be same as $LR(0)$ table.

Note : If the grammar is LR(0) it will be SLR(1) also but if it is ~~LR(0)~~ not LR(0) then it ~~SLR(1)~~ may or may not be SLR(1).

Ques 1: $G: S \rightarrow AaAb \mid BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

Given this grammar, find out if this is LR(0) and SLR (1) or not.

Ques 2: Given the grammar, $G: A \rightarrow (A) | a$
parse the input string using shift reduce parsing technique
 $| P: ((a))$



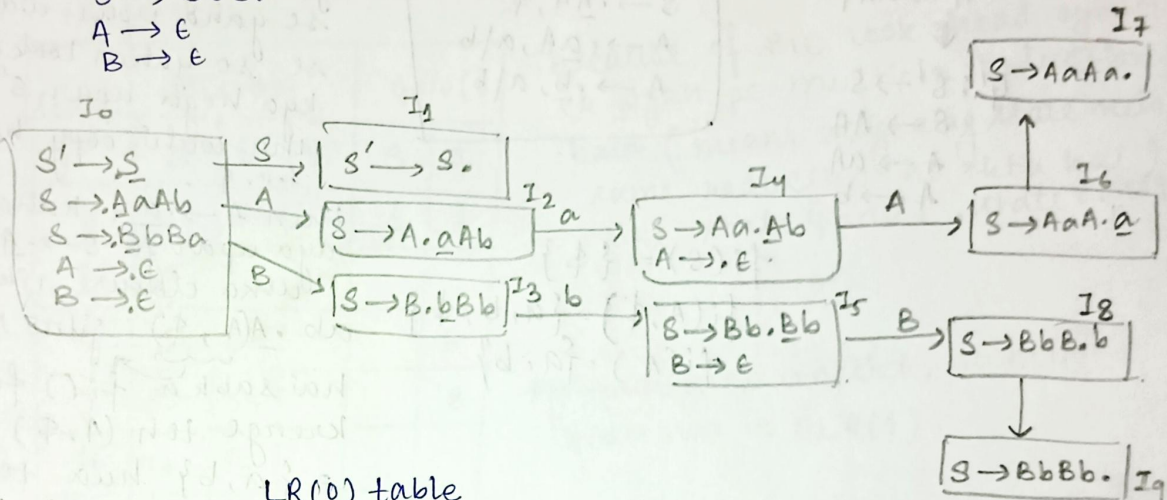
31/03/26

Ques:

$G: S \rightarrow AaAb \mid BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

Given this grammar, find out if this is LR(0) and SLR(1) or not.

$G': S' \rightarrow S$
 $S \rightarrow AaAb$
 $S \rightarrow BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$



LR(0) table

state no.	action			GoTo		
	a	b	\$	S	A	B
0	r3/r4	r3/r4	r3/r4	1	2	3
1			accept			
2	s4					
3		s5				
4					6	
5						8
6		s7				
7	r1	r1	r1			
8	s9					
9	r2	r2	r2			

The grammar is not LR(0) as it has RR conflict.

$$fo(S) = \$$$

$$fo(A) = \{a, b\}$$

$$fo(B) = \{a, b\}$$

CLR(1)

look ahead symbol
Conical LR(1)

Ques: $G: S \rightarrow AA$
 $A \rightarrow aA \mid b$
 $\downarrow$
 $G': S' \rightarrow S$
 $S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

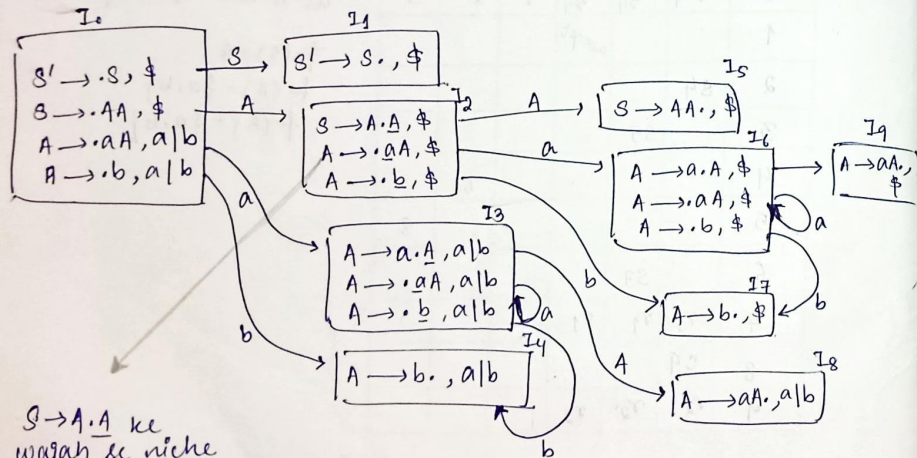
I_0

$S' \rightarrow \cdot S, \$$
$S \rightarrow \cdot AA, \$$
$A \rightarrow \cdot aA, a b$
$A \rightarrow \cdot b, a b$

$fi(S) = \{ \$ \}$
 $fi(A, \$) = \{ a, b, \$ \}$
 $fi(A) = \{ a, b \}$

$S' \rightarrow \cdot S \rightarrow \cdot$ ke baad jo v
hoga uska $fi()$ find out
krenge. $\cdot S$ ka $fi()$ $\$$ hai
then, $S \rightarrow \cdot AA$ hai toh
yahan ye closure kahan
se yahd upar wale $S' \rightarrow \cdot S$
se so yahan lookahead
kya hoga upar S' mein jo
tha wohi copy ho jayega.
i.e. $\$$.

then $A \rightarrow \cdot aA$ kahan se
aya upar jo $S \rightarrow \cdot AA$ tha
uska closure likhe so
ab $\cdot A(A, \$)$ jitna remaining
hai sabka $fi()$ find out
krenge toh $(A, \$)$ ka $fi()$
 $= \{ a, b \}$ hua toh
lookahead mein $a|b$
likhenge same goes for
niche wala like a woh
upar $S \rightarrow \cdot AA$ ka closure hai.



$S \rightarrow A \cdot A$ ke
wajah se niche
uska closure $A \rightarrow \cdot aA$
 $A \rightarrow \cdot b$
aya toh upar jo $fi()$
nikla tha niche v
wahi sahega

CLR(1) keypoints:-

- Place all the reduction entries in the look ahead symbols only.
- Lookahead likhna hai ab mein dhyan se upar wale ko check krke.

State no.	action			GoTo()	
	a	b	$\$$	S	A
0	S3	S4		1	2
1			accept		
2	S6	S7			5
3	S3	S4			8
4	r3	r3			
5			r1		
6	S6	S7			9
7			r3		
8	r2	r2			
9			r2		

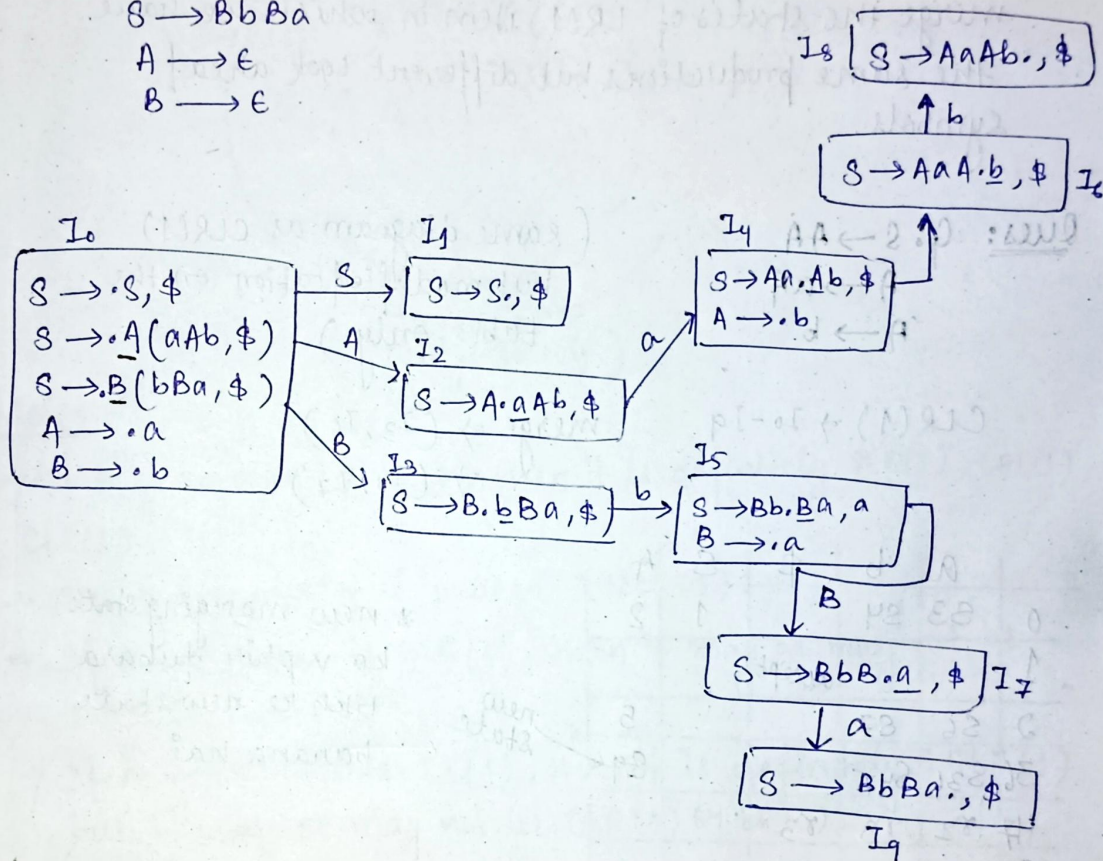
because of the lookahead symbol
ek jagah pe multiple reduction
hain (means alag alag state mein
same reduction ho skta hai)
like state 4 and 7, state 8 and 9.

As there is no conflict, so the
grammar is CLR(1).

NOTE: If a grammar is not LR(0)
or SLR(1) then it can be
CLR(1).

Ques: $G: S \rightarrow AaAb \mid BbBa$ find out CLR(1) or not.
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

$G': S' \rightarrow S$
 $S \rightarrow AaAb$
 $S \rightarrow BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$



	a	b	ϕ	S	A	B
0	r3	r4		1	2	3
1			accept			
2	s4					
3		s5				
4		r3			6	
5	r4				.	7
6		s8				
7	s9					
8			r1			
9			r2			

there is no conflict in the table.
 So, the grammar is CLR(1).

LALR(1) parser

→ Look ahead LR(1)

This is minimized version of CLR(1). In this, we all merge the states of LR(1) item in which we have the same productions but different look ahead symbols.

Ques: $G: S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

(same diagram as CLR(1) but modification on the table only)

CLR(1) $\rightarrow I_0 - I_9$

merge $\rightarrow (I_3, I_6)$
 $\rightarrow (I_4, I_7)$

	a	b	\$	S	A
0	S3	S4		1	2
1			accept		
2	S6	S7			5
3	S36	S47			89 ← new state
4	r2	r3	r3		
5			r1		
6	r2	r2	r2		

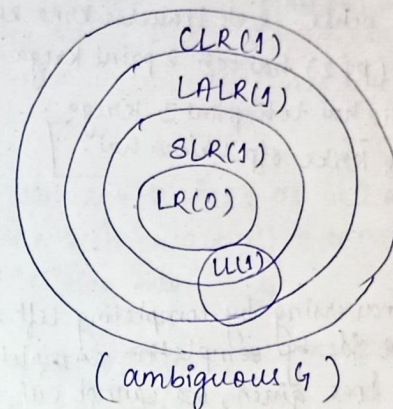
* new merging state ko v phir dubara likh ke new state banana hai

LALR(1) = 7 states

no. of states (n):

$$n_{LR(0)} = n_{SLR(1)} = n_{LALR(1)} \leq n_{CLR(1)}$$

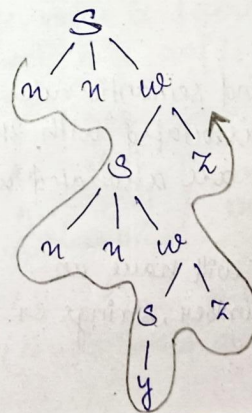
Relationship between Parser:



Note:

- If a grammar is LR(0) then it is definitely SLR(1), LALR(1), CLR(1).
- the most powerful parser is CLR(1).
- If a grammar is SLR(1), then it may or may not be LR(0).
- If a grammar is LL(1), then it is definitely LALR(1) but it may or may not be SLR(1) or LR(0).
- If a grammar is LALR(1), then it is definitely CLR(1).

Intro to SDD (Syntax Directed Definition)



Semantic actions

- 1) $S \rightarrow y$ (print 2)
- 2) $w \rightarrow sz$ (print 3)
- 3) $S \rightarrow nnnw$ (print 1)

Given this parse tree, find out the output.

Op: 23131

[pehle ek ek traverse krte hue jayega jo completely mil jayega usko print krega.
 As yahan pe sabse pehle ek ek traverse krne ke baad y mila toh $S \rightarrow y$ (pf 2) hai toh 2 print krega then $w \rightarrow Sx$ complete ho rha hai toh print 3 krega.
 aise hi sare traverse krke off vikhna hai.]

Note:

→ We have to start traversing by completing left side then moving to right side by completing completely depending upon the tree given, we cannot cut in between.

Ques: $G: S \rightarrow S \# T$
 $S \rightarrow T$
 $T \rightarrow T \% R$
 $T \rightarrow R$
 $R \rightarrow id$

$\{s.val = s.val * T.val\}$
 $\{s.val = T.val\}$
 $\{T.val = T.val \div R.val\}$
 $\{T.val = R.val\}$
 $\{R.val = id.val\}$

ip $20 \# 10 \% 5 \# 8 \% 2 \% 2$ (BODMAS follow krega)
 $= 20 * 2 * 2$
 $= 40 * 2$
 $= 80$

06/04/26

SDD:

→ It is a combination of grammar and semantic rules.
 There are attributes which are associated with the grammar and the semantic rules are associated with the production rules.

→ Each non terminal or variable will have an attribute value that may be a number, string or datatype.

Types of SDD:

→ S-Attributed SDD
 → L-Attributed SDD

① It uses only synthesized attribute

② The semantic rules or actions are placed at the extreme right end of any prod. rule.

③ Attributes are evaluated with bottom up power.

① It uses both synthesized as well as inherited attribute but with a restriction that it can inherit from parent or left sibling only.

② The semantic rules or actions are placed anywhere on the right hand side of the prod. rule (at the beginning, centre or extreme right end).

③ Attributes are evaluated by traversing the parse tree in depth first manner from left to right.

Synthesized attribute:

An attribute is said to be synthesized attribute if the node value is determined by the attribute value of its child nodes.

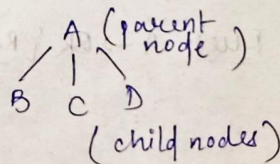
Inherited attribute:

An attribute is said to be inherited attribute if the node value is determined by the attribute value of parent node and/or sibling node.

eg: $A \rightarrow BCD$

$A.i \rightarrow B.i$
 $A.i \rightarrow C.i$
 $A.i \rightarrow D.i$

} semantic rules/actions



All of them are synthesized attribute.
 (S-attributed SDD because all are synthesized attribute)

eg: $A \rightarrow BCD$

$c.i = A.i$
 $c.i = B.i$
 $c.i = D.i$

inherited
 semantic rules

The value of node c is determined by parent node 'A'.
 (inherited attribute)

Even though all the semantic rules are inherited attributes, it is not L-attributed SDD because node c is taking attributed value from the right sibling D which is not allowed. So, it is not L-attributed SDD.

Ques: $A \rightarrow BC$ $\{B.i = A.i, c.i = B.i, A.i = c.i\}$
 Given the question, find out if it is S-attributed or L-attributed SDD.

$B.i = A.i$
 inherited

$c.i = B.i$
 inherited

$A.i = c.i$
 synthesized

$A \rightarrow \{B.i = A.i\} BC \{ \dots \}$



value of B is determined by parent node A (L-attributed SDD because it is using both synthesized & inherited attribute)

Ques: $A \rightarrow QR$ $\{R.i = A.i, Q.i = R.i, A.i = Q.i\}$
 inherited (right sibling) synthesized

neither S-nor L-attributed SDD (because inherited attribute is using value from right sibling which is not allowed).

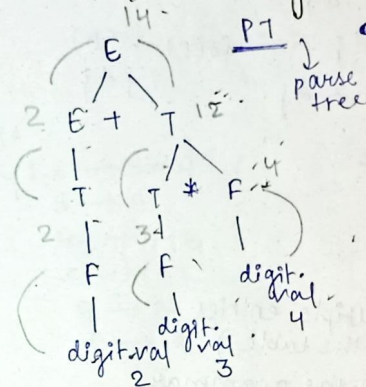
Ques: $G: E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow \text{digit.val}$

semantic rules

$\{B.val = E.val + T.val\}$
 $\{E.val = T.val\}$
 $\{T.val = T.val * F.val\}$
 $\{T.val = F.val\}$
 $\{F.val = \text{digit.val}\}$

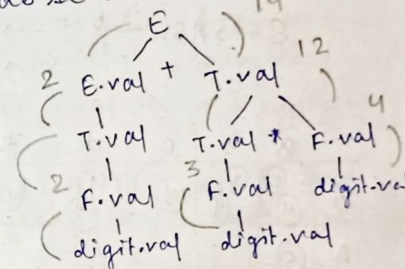
exp: $2 + 3 * 4$

Given the grammar and the semantic rules, evaluate the given expression using tree.



dependency graph

ek ek semantic rules ke hisab se banana hai.



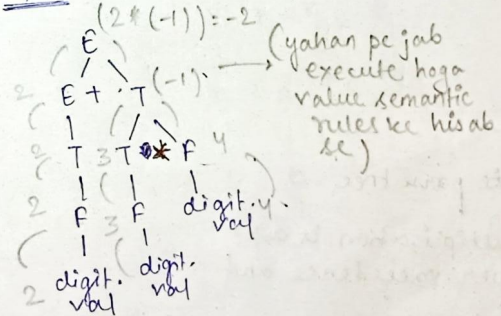
Hw

Ques: $G: E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow \text{digit.val}$

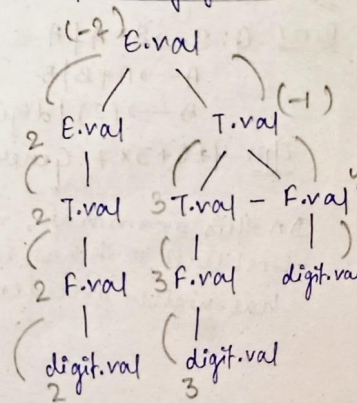
semantic rules

$\{E.val = E.val * T.val\}$
 $\{E.val = T.val\}$
 $\{T.val = T.val - F.val\}$
 $\{T.val = F.val\}$
 $\{F.val = \text{digit.val}\}$

PT



dependency graph



Ques: $G: S \rightarrow aBA \mid aBa \mid bA \mid C$
 $A \rightarrow aB \mid dA$
 $B \rightarrow CB \mid aBd$
 $C \rightarrow Cd \mid e$
 $D \rightarrow d$

$\{a, b, c, d, e\}$	$\{a, b, c, d\}$
$\{a, d\}$	$\{a, b, c, d\}$
$\{c, a, d\}$	$\{a, b, c, d\}$
$\{c, e\}$	$\{a, b, c, d\}$
$\{d\}$	$\{a, b, c, d\}$

$$\begin{aligned} f_0(s) &= f_0(A) = \{\$, a, b, c, d\} \\ f_0(A) &= f_1(s) = \{a, b, c, d\} \\ f_0(A) &= f_0(s) = \{\$, a, b, c, d\} \end{aligned} \quad \begin{aligned} f_0(c) &= f_1(c) \\ &= \{d\} \end{aligned}$$
$$S \rightarrow ASA \rightarrow S \rightarrow AEA$$
$$S \rightarrow Aa$$

	a	b	c	d	e
S	1 2 4	3 4	4	1 4	4
A	5			6	
B	7 8	7 ^x	7	7	7
C	10	10	10 11	10	10
D				11	

(multiple entries in the table)

So, the grammar is not LL(1).

09/04/26

Ques $G: S \rightarrow S * A \mid A$
 $A \rightarrow A + B \mid B$
 $B \rightarrow (S) \mid \text{digit}$

i/p: $4 * 6 + 3 * 7$ Generate parse tree

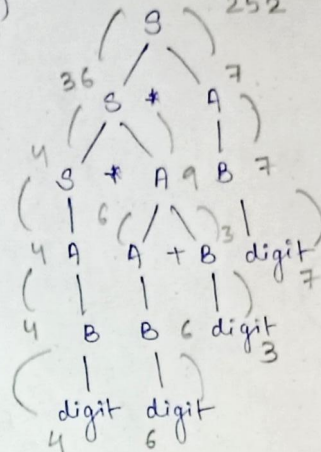
In this grammar, multiplication is at first level so it has lower precedence and has higher precedence.

fic)

fol

(PT)

252



Query:

$$\begin{aligned} G: S &\rightarrow S^* A \\ S &\rightarrow A \\ A &\rightarrow A + B \\ B &\rightarrow (S) \\ A &\rightarrow B \\ B &\rightarrow \text{id} \end{aligned}$$

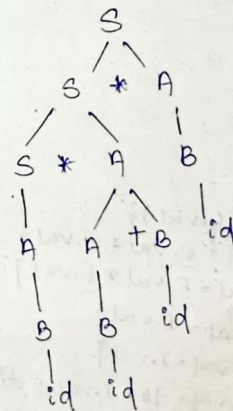
semantic rules

$\{S.val = S.val * A.val\}$ synthesized
 $\{S.val = A.val\}$ synthesized
 $\{A.val = A.val - B.val\}$ synthesized
 $\{B.val = 2\}$
 $\{A.val = B.val\}$ synthesized
 $\{B.val = id.val\}$ synthesized

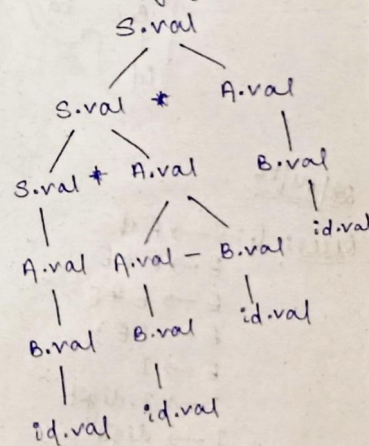
Given this grammar, find out if it is S-attributed or L-attributed SDD. (S-attributed SDD)

i/p: $4 * 6 + 3 * 7$

pause free

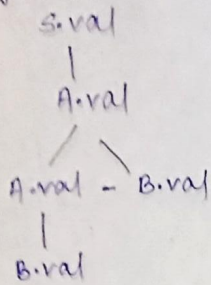


dependency graph

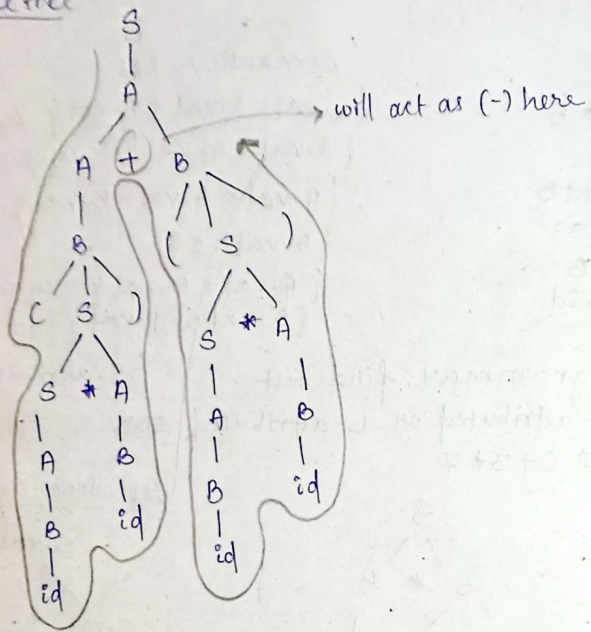


IP: $(4 \times 6) + (3 \times 7)$

dependency graph



parse tree



28/04/26

Ques: $G: S \rightarrow E \mid$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $E \rightarrow I$
 $I \rightarrow I.\text{digit}$
 $I \rightarrow \text{digit}$

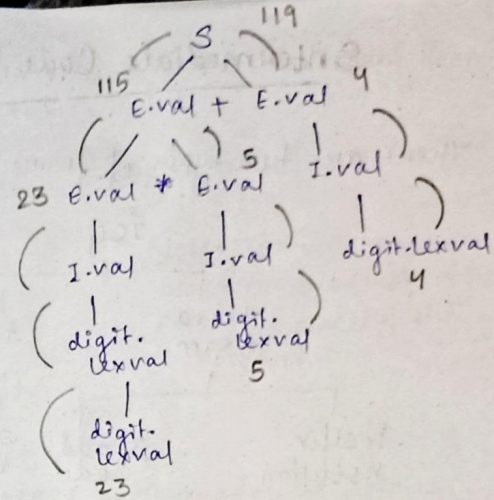
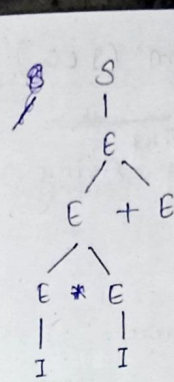
```

{ print (E.val) }
{ E.val = E.val + E.val }
{ E.val = E.val * E.val }
{ E.val = E.val }
{ E.val = I.val }
{ I.val = 10 * I.val + digit.lexval }
{ I.val = digit.lexval }

```

ip: 23 * 5 + 4 \$

Given the SDD, you have to find out the value of the root node by constructing the dependency graph.



Ques:

$$G: s \rightarrow \text{newly}$$

$$w \rightarrow \text{sz}$$

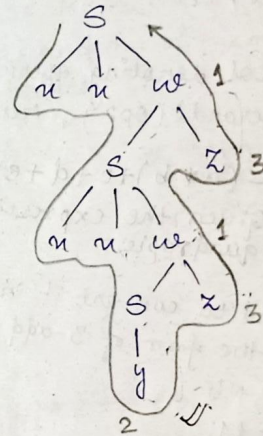
semantic rules

$$\{pf(1); pf(2)\}$$

$$\{pf(3)\}.$$

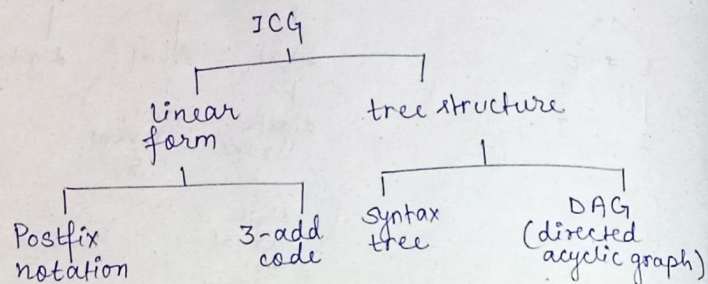
ip: nnnnyzz

off : 23131



Intermediate Code Generation (ICG)

There are two types of forms for ^{deriving} ~~string~~ a string.



Representation of 3-add code:

- (1) Quadruples
- (2) Triples
- (3) Indirect Triples

Quadruples: 4 col denoting operator, operand1 (op1), operand2 (op2), result.

ex: $-(a * b) + c + d + e$
Given the expression, represent in quadruple.

first we need to convert it into 3-add code because it is not in the form of 3-add code.

$$t_1 = a * b$$

$$t_2 = -t_1$$

$$t_3 = c + d$$

$$t_4 = t_3 + e$$

$$t_5 = t_4 + t_2 \rightarrow [-(a * b) + c + d + e]$$

operator	op1	op2	result	Quadruple Table
(0) *	a	b	t1	Quadruple Table ↙
(1) -	t1	-	t2	
(2) +	c	d	t3	
(3) +	t3	e	t4	
(4) +	t4	t2	t5	

→ reference no.

Triples: 3 col - the result column will be removed from the quadruples table.

operator	op1	op2	Triples table
*	a	b	Triples table ↙
-	(0)		
+	c	d	
+	(2)	e	
+	(3)	(1)	

Note: we will use reference no. from quadruple table here in places of the operand.

Hw

Ques: Write down the difference betn quadruples, triples and indirect triples.

30/04/26

Indirect Triples: 2-cols: In indirect triples, we will have 2 memory references and one of them will be taken from the triples tables.

memory address	reference number	ye reference number quadruples or triples ko point out kr rha hai.
100	(0)	
101	(1)	
102	(2)	
103	(3)	
104	(4)	

Hw

Ques expression: $x + (-y) * x + a / b$

Given the expression, you have to construct quadruples, triples and indirect triples.

$$t_1 = -y * x$$

$$t_2 = a / b$$

Single Static Assignment (SSA)

It is a type of 3-address code where all assignments to the variables are with distinct names. It means no variable will be reassigned again.

ex: $p = a + b$ $\xrightarrow{SSA}$ $p_1 = a_1 + b_1$
 $q = p - c$ $\xrightarrow{SSA}$ $q_1 = p_1 - c_1$
 $p = q * d$ $\xrightarrow{SSA}$ $p_2 = q_1 * d_1$
 $p = e - p$ $\xrightarrow{SSA}$ $p_3 = e_1 - p_2$ $\rightarrow$ latest update value p_2 hai toh $p_3 = e_1 - p_2$ hua.
 $q = p + q$ $\xrightarrow{SSA}$ $q_2 = p_3 + q_1$

Ques: $x = u - t$ $\xrightarrow{SSA}$ $x_1 = u_1 - t_1$
 $y = x * v$ $\xrightarrow{SSA}$ $y_1 = x_1 * v_1$
 $u = y + w$ $\xrightarrow{SSA}$ $u_2 = y_1 + w_1$
 $y = t - x$ $\xrightarrow{SSA}$ $y_2 = t_1 - x_1$
 $y = x * y$ $\xrightarrow{SSA}$ $y_3 = x_2 * y_2$
 $\rightarrow$ total 10 variables.

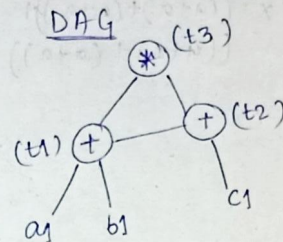
Directed Acyclic Graph (DAG):

$\rightarrow$ Directed acyclic graph is used to represent the structure of basic blocks to visualize the flow of values b/w those basic blocks.

$\rightarrow$ It helps in detecting the common sub expression and avoids recomputation for the same.

expression: $(a * + b) * (a + b + c)$

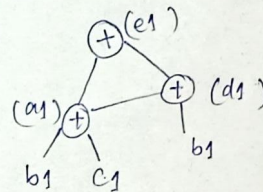
$t_0 = a + b$ $\xrightarrow{SSA}$ $t_0 = a_1 + b_1$
 $t_1 = t_0 + c$ $\xrightarrow{SSA}$ $t_1 = t_0 + c_1$
 $t_2 = t_0 * t_1$ $\xrightarrow{SSA}$ $t_2 = t_0 * t_1$



pehle $a_1 + b_1$ ka banaenge operator + leke aur side mein jahan store ho rha hai woh likhenge then usi mein c add ho rha hai toh wahise leke + operator krke c likh denge then dono * ho rha hai toh woh t_3 mein store krdenge.

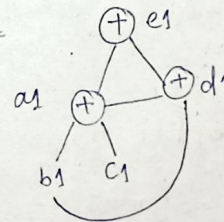
Ques: $a = b + c$
 $d = b + a$
 $e = d + a$ $\xrightarrow{SSA}$

$a_1 = b_1 + c_1$
 $d_1 = b_1 + a_1$
 $e_1 = d_1 + a_1$



total = 6 nodes

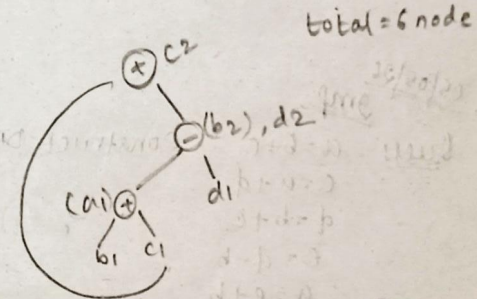
OR



Ques: $a = b + c$
 $b = a - d$
 $c = b + c$
 $d = a - d$

$\xrightarrow{SSA}$ $a_1 = b_1 + c_1$
 $b_2 = a_1 - d_1$
 $c_2 = b_2 + c_1$
 $d_2 = a_1 - d_1$

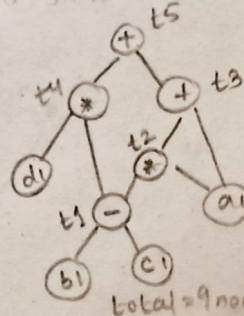
common b+c hai toh b1 nahlega



total = 6 node

HW Ques: $(a + a * (b - c)) + ((b - c) * d)$

$t_1 = b - c$ $\xrightarrow{SSA}$ $t_1 = b_1 - c_1$
 $t_2 = a * t_1$ $\xrightarrow{SSA}$ $t_2 = a_1 * t_1$
 $t_3 = a + t_2$ $\xrightarrow{SSA}$ $t_3 = a_1 + t_2$
 $t_4 = d * t_1$ $\xrightarrow{SSA}$ $t_4 = d_1 * t_1$
 $t_5 = t_3 + t_4$ $\xrightarrow{SSA}$ $t_5 = t_3 + t_4$



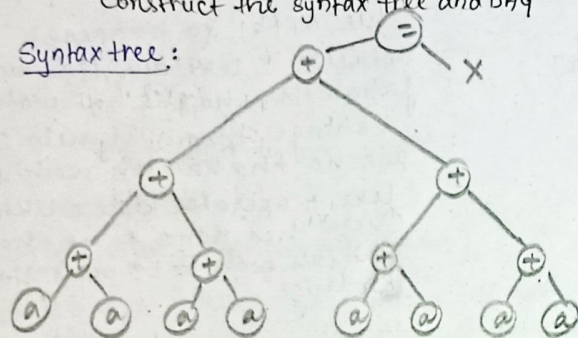
total = 9 node

where common sub expression $(b-c)$ is being reused along with node a_1 .

04/05/26

Ques: Given the 3-add expression convert Construct the syntax tree and DAG
 $x = ((a+a) + (a+a)) + ((a+a) + (a+a))$

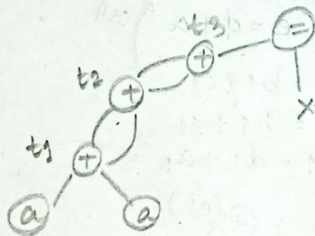
Syntax tree:



total = 15 node

DAG:

$t_1 = a + a$
 $t_2 = t_1 + t_1$
 $t_3 = t_2 + t_2$



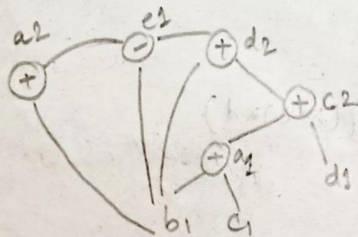
total = 8 nodes

05/05/26 gmp

Ques: Construct DAG

$a = b + c$
 $c = a + d$
 $d = b + c$
 $e = d - b$
 $a = e + b$

$a_1 = b_1 + c_1$
 $c_2 = a_1 + d_1$
 $d_2 = b_1 + c_2$
 $e_1 = d_2 - b_1$
 $a_2 = e_1 + b_1$

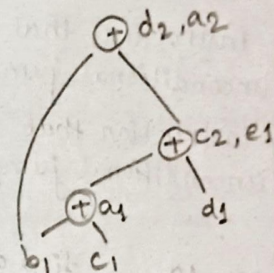


OR

$a = b + c$
 $c = a + d$
 $d = b + c$
 $e = d - b$
 $a = e + b$

SSA $\Rightarrow$

$a_1 = b_1 + c_1$
 $c_2 = a_1 + d_1$
 $d_2 = b_1 + c_2$
 $e_1 = c_2$
 $a_2 = d_2$



Note:

Whenever question is asked for minimum no. of node and edges, first simplify the 3-add code to reduce the no. of variables then convert it into SSA and then DAG.

Basic Blocks and Flow Graph

- While doing optimization, we have to find out the loops and optimize them in our code.
- For detecting loops, we will use control flow analysis using the program flow graph.
- To find the program flow graph, we need to find basic blocks.
- A basic block is a sequence of 3-add statement where control enters at the beginning and leaves at the end without any jumps or halts.
- In order to find out the basic block, we need to find the leaders in the 3-add code.
- A basic block will start from 1 leader to the next leader but not including the next leader.

Finding a leader in a 3-add code:

- The first statement of 3-add instruction in the intermediate code is a leader.
- Any instructions that is the target of conditional or unconditional jump is a leader.
- Any instruction that immediately follows a conditional or unconditional jump is a leader.

Ques: * L1 find out the ^{no. of} basic blocks and draw the flow graph.

BB1 [1. $x = 10$
2. $y = 20$
3. $z = 0$
4. if ($x > y$) goto (7) → conditional goto
→ this is known as target

BB2 [5. $z = x - y$
6. goto 8 → unconditional target/goto

BB3 - 12 * 7. $z = x + y$

BB4 - 13 * 8. result = $z * 2$

The first line in the instruction is a leader by default i.e. $x = 10$.

no. of leaders = 4 (L1, L5, L7, L8) → these lines are the leaders

[conditional statement ka target 7 hai toh line 7 will be a leader then line 6 mein target (8) hai toh line 8 will be a leader then conditional statement ka immediate niche wala line i.e. line 5 woh ek leader hai]

Basic blocks BB1 = line (1-4)

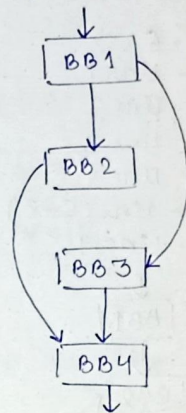
BB2 = (5-6)

BB3 = line 7

BB4 = line 8

07/05/26

flow graph:



BB1 mein condn statement hai jo ki goto (7) kr rha hai to woh BB3 se connected hai.
BB2 result line ko ultimately jayega toh isliye BB4 se connected hai.

Ques: find out the no. of leaders and basic blocks.

- * 1. $f = 1$] BB1
- 2. $i = 2$
- * 3. if ($i > n$) goto (8) - BB2
- * 4. $t1 = f * i$] BB3
- 5. $f = t1$
- 6. $i = i + 1$
- 7. goto (3)
- * 8. goto calling function $f(n)$ - BB4

Total leaders = 4 i.e. (L1, L3, L4, L8) → these lines are the leaders

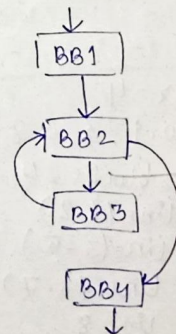
Basic blocks = 4 i.e. BB1 - line (1-2)

BB2 - line 3

BB3 - line (4-7)

BB4 - line 8

flow graph:



else part hai isliye BB2 and BB3 connected hain

Ques:

- * 1. $i = 7$
- * 2. if ($i \leq 923$) goto(5)
- * 3. goto(9)
- * 4. $t1 = b * c$
- * 5. $t2 = a + t1$
6. $x = t2$
7. $i = i + 1$
8. goto(2)
- * 9. end

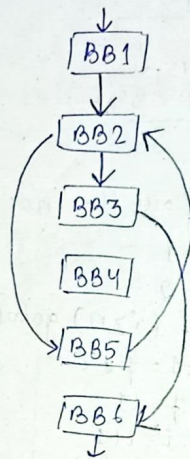
total leaders = 6
i.e. Line 1, 2, 3, 4, 5, 9

Basic blocks = 6
i.e. BB1 - Line 1
BB2 - Line 2
BB3 - Line 3
BB4 - Line 4
BB5 - Line (5-8)
BB6 - Line 9

note:

[no. of nodes = no. of basic blocks
no. of edges will include both entry and exit point as well as all the connections from basic blocks.]

flow graph



11/05/26

Code Optimization:

Ques:

```

if (a+b > c) {
    d = a+b*c;
}
else {
    d = a-b;
}
  
```

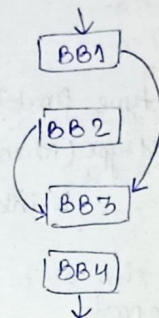
Give this code, you have to find out the total no. of basic blocks and construct the control flow graph.

- * 1. $t1 = a+b$
- * 2. if $t1 > c$ goto(6)
- * 3. $t2 = a-b$
- * 4. $d = t2$
- * 5. goto(8)
- * 6. $t3 = t1 * c$
- * 7. $d = t3$
- * 8. end

total leader = 4
i.e. Line no. 1, 3, 6, 8

total basic blocks = 4
i.e. BB1 - Line (1-2)
BB2 - Line (3-5)
BB3 - Line (6-7)
BB4 - Line 8

flow chart



→ There are 2 types of code optimization.

- (1) machine dependent
- (11) machine independent

Machine independent:

→ It does not involve any CPU registers or any absolute memory locations.

Machine dependent:

→ This optimization is done after the target code is generated.

→ It transforms the code according to the target machine architecture.

→ It involves CPU registers as well as absolute memory locations.

There are few examples of machine independent:

Imp (1) Loop optimization

- (a) code motion or freq. reduction
- (b) loop unrolling
- (c) loop jamming

There are other methods.

machine dependent examples:

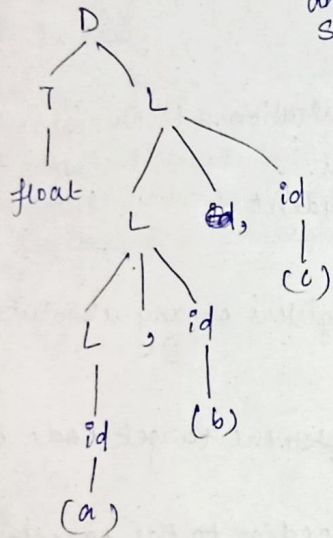
(1) peephole optimization.

12/05/26

Ques: $D \rightarrow TL$
 $T \rightarrow \text{int} \mid \text{float}$
 $L \rightarrow L, \text{id} \mid \text{id}$

inp: float a, b, c

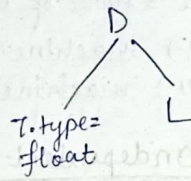
parse tree



semantic rules.

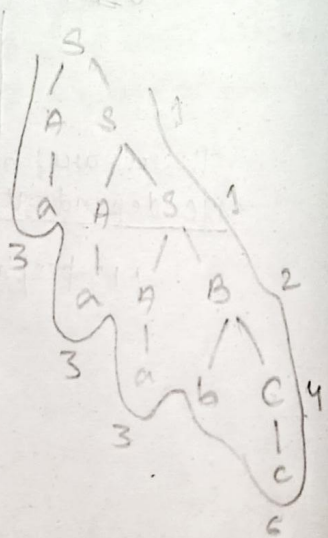
$\{L.inh = T.type\}$
 $\{T.type = \text{int}\} \mid \{T.type = \text{float}\}$
 $\{L.inh = L.inh\} \mid \{add_type(id.entity, L.inh)\}$

annotated parse tree is similar to SDD ka dependency graph.



Ques: $S \rightarrow AS$ $\{pf(1)\}$ inp: aaabbc
 $S \rightarrow AB$ $\{pf(2)\}$
 $A \rightarrow a$ $\{pf(3)\}$
 $B \rightarrow bc$ $\{pf(4)\}$
 $B \rightarrow CB$ $\{pf(5)\}$
 $C \rightarrow c$ $\{pf(6)\}$

o/p: 33364211



Ques: $E \rightarrow E, \#T$
 $E \rightarrow T$
 $T \rightarrow T_1 \& F$
 $T \rightarrow F$
 $F \rightarrow \text{num}$

$\{E.val = E_1.val \# T.val\}$
 $\{E.val = T.val\}$
 $\{T.val = T_1.val \& F.val\}$
 $\{T.val = F.val\}$
 $\{F.val = \text{num}\}$

inp: 8#2&4#16&12#4&2

o/p: -128

Construct the dependency graph as well as fill the gap in the question mark